

NATIVX ASSISTANT

Diagramele arhitecturii, explicate pe înțelesul tuturor

Ce face fiecare diagramă, de ce am construit sistemul așa
și care e logica din spate — pas cu pas, fără jargon

Generat: 10 august 2026

Sursa: docs/ARCHITECTURE-WORKFLOWS.md — revizia auditată 6bbeb6f

Fiecare săgeată din diagrame corespunde unui apel real din cod, verificat

Cum citești acest document

De ce sunt 14 diagrame și nu una singură?

Pentru că o singură diagramă „cu tot sistemul” ar fi ori **falsă** (ar ascunde lucruri), ori **ilizibilă** (o pânză de păianjen cu sute de săgeți). Sistemul are mai multe „axe” independente: ce procese rulează, cum circulă un mesaj, cum se caută în catalog, ce se întâmplă la erori. Fiecare axă are diagrama ei, iar împreună acoperă tot.

Diagramele sunt organizate ca un **zoom progresiv** — de la harta lumii până la strada ta:

Nivel de zoom	Întrebarea la care răspunde	Diagrama
1 • Context	Cine vorbește cu sistemul și de ce servicii externe depinde?	Diagrama 1
2 • Procese	Ce programe rulează și cum pornesc ele în siguranță?	Diagrama 2
3 • Un mesaj	Intră un mesaj — ce se întâmplă cu el, pas cu pas?	Diagrama 3
4 • Creierul	Cum se decide ce răspuns primește clientul?	Diagramele 4a, 4b, 4c
5 • Subiecte	Căutare, memorie, baza de date, servicii externe, erori, mesaje proactive	Diagramele 5–10

Legenda culorilor (valabilă, cu mici variații, în toate diagramele)

Culoare	Ce înseamnă
 Galben	O decizie — o întrebare cu „da / nu” din care fluxul se ramifică.
 Verde	Un pas de cod — determinist, previzibil, fără inteligență artificială. Rulează la fel de fiecare dată.
 Mov	Un apel la AI — costă bani și poate greși; de aceea tot ce iese de aici e verificat de cod.
 Turcoaz	Un drum gratuit — răspuns servit fără niciun apel la AI (sau un proces de fundal).
 Roșu / roz	Eroare, pericol, filtru de siguranță sau coada de mesaje — locurile unde sistemul refuză, protejează sau semnalează.
 Albastru	O margină — punct de intrare sau de ieșire spre lumea exterioară (sau răspunsul final).
 Gri	Stocare — baza de date sau un tabel anume.

Trei lucruri de știut înainte să începi

- Diagramele sunt vectoriale.** Dacă o diagramă pare mică pe pagină, dă zoom în PDF — textul rămâne perfect clar la orice mărire, nu se pixelează.
- Nimic nu e desenat „din burtă”.** Fiecare căsuță are în paranteză o citare de forma `fișier:linie` — locul exact din cod care face acel pas. Documentul-mamă e verificat automat la fiecare modificare de cod (un script compară listele din document cu codul real; dacă diverg, verificarea pică și documentul trebuie reparat).
- Structura fiecărui capitol e aceeași:** „Ce arată” (pe scurt), „De ce am construit-o așa” (decizia de arhitectură), „Logica, pas cu pas” (plimbarea prin diagramă) și „De reținut” (ideea cu care să rămâi).

Mic dicționar — termenii care apar în diagrame

Termen	Pe românește
Pipeline	Banda rulantă a sistemului: un mesaj trece printr-un șir fix de stații (numite „stagii”), în aceeași ordine, mereu. Orice stație poate produce răspunsul final și atunci restul se sare.
Webhook	„Soneria” noastră pe internet: adresa la care Meta (WhatsApp) sau platforma de magazin ne „sună” când s-a întâmplat ceva (a venit un mesaj, s-a plasat o comandă).
Redis / coadă / stream	O memorie foarte rapidă folosită ca bandă transportoare: recepția pune mesajele pe bandă, muncitorii le iau de pe bandă când au timp. Așa recepția nu așteaptă niciodată după bucatărie.
Worker (muncitorul)	Programul care ia mesajele de pe bandă și le procesează efectiv — el rulează pipeline-ul.
Outbox + Dispatcher (cutia de ieșire + dispecerul)	Tot ce pleacă spre client se scrie mai întâi într-un tabel („outbox” = cutia de scrisori de trimis), iar un program separat („dispecerul”) le ia de acolo și le trimite efectiv. E singurul punct de ieșire din sistem.
LLM / AI „nano” și „mini”	Modelele de inteligență artificială folosite. „Nano” = cel mic și ieftin (sortează mesajele, scrie rezumate). „Mini” = cel mare (vânzătorul propriu-zis: caută în catalog, compune recomandarea).
Triaj	Sortarea de la urgențe: AI-ul mic citește mesajul și decide pe ce ușă merge — vânzare, comandă, întrebare simplă, sau „nu e clar, întreabă”.
Embedding / căutare semantică	Transformarea unei fraze într-un șir de numere care îi codifică înțelesul. Așa putem găsi produse „apropiate ca sens”, chiar dacă nu conțin cuvintele exacte din întrebare.
Cache semantic	Memoria de răspunsuri: dacă am mai răspuns la o întrebare aproape identică, servim răspunsul salvat — instant și gratis, fără AI.
Tenant / multi-tenant	Un „tenant” = un magazin client. Platforma servește mai multe magazine din același sistem, iar datele lor trebuie să fie perfect separate — asta e „multi-tenant”.
RLS (Row-Level Security)	Polițistul din baza de date: reguli care fac ca fiecare interogare să vadă <i>numai</i> rândurile magazinului ei. Chiar și un query greșit primește „0 rezultate”, nu datele altui client.
Dedupe (deduplicare)	Protecția anti-dubluri: Meta re-trimite același mesaj de mai multe ori dacă nu primește confirmare rapid. Fără dedupe, clientul ar primi același răspuns de 2–3 ori.
Debounce	Pauza de politețe: oamenii scriu în rafale („bună” ... „am o întrebare” ... „aveți șampon?”). Așteptăm 3 secunde de liniște și lipim rafala într-un singur mesaj — un singur răspuns, nu trei.
SSE	Canalul prin care serverul „împinge” mesaje către pagina web a clientului, în timp real, fără ca pagina să tot întrebe „a venit ceva?”.
Handoff	Predarea ștafetei către un om: botul tace, un operator uman preia conversația.
Tranzacție (TX)	Un pachet de scrieri în baza de date care reușește <i>ori tot, ori deloc</i> . Nu există stare intermediară de tip „am salvat mesajul dar am pierdut răspunsul”.

Fallback	Planul de rezervă: ce facem când planul A eșuează. Regula casei: <i>niciodată tăcere</i> — clientul primește mereu ceva.
Flag / kill-switch	Un întrerupător de funcționalitate: fiecare mecanism important poate fi pornit/oprit din configurație, fără modificări de cod.
Idempotent	O operațiune pe care o poți repeta fără efecte duble: dacă programul crapă și reia lucrul, mesajul tot o singură dată se trimite.

Diagrama 1 — Harta mare: tot sistemul dintr-o privire

Ce arată

Aceasta e „harta lumii”: toate piesele mari și cum sunt legate între ele. De la stânga la dreapta: **ușile de intrare** (clienți pe WhatsApp, Telegram, widgetul de pe site, plus platforma de magazin care ne anunță comenzile), **coada Redis** din mijloc, **muncitorul** care gândește (worker + pipeline), **ieșirea** (outbox → dispecer → canale), **procesele de fundal** (joburi de noapte și mesaje proactive) și cele două dependențe mari: **baza de date Postgres** și **OpenAI**.

ANALOGIE

Gândește-te la un restaurant: **recepția** (webhook) doar preia comanda și o pune pe șină — nu gătește nimic; **bucătăria** (worker) gătește în ritmul ei; **un singur ospătar** (dispecerul) duce farfuriile la mese; iar **contabilul** (joburile de fundal) lucrează noaptea, când nu deranjează pe nimeni. Fiecare își vede de treaba lui și nu se calcă pe picioare.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

- 1. Margini subțiri.** WhatsApp (Meta) cere confirmare în sub 50 de milisecunde, altfel re-trimite mesajul agresiv. De aceea recepția nu atinge baza de date deloc: verifică semnătura, pune mesajul pe coadă, confirmă. Orice muncă grea acolo ar produce avalanșe de dubluri.
- 2. Coada din mijloc decuplează viteza de intrare de viteza de gândire.** Pot intra 100 de mesaje pe secundă; bucătăria le procesează în ritmul ei, fără să piardă nimic.
- 3. Un singur punct de ieșire.** Nimeni nu trimite mesaje „direct” — totul trece prin outbox și dispecer. Așa avem un singur loc pentru reîncercări la eșec, evidența a tot ce s-a trimis, și zero riscul ca două bucăți de cod să trimită dubluri.

Logica, pas cu pas

1. Un client scrie pe WhatsApp / Telegram / site → mesajul intră pe una din ușile albastre.
2. Ușa face verificările minime (semnătură, dubluri) și pune un plic *neutru* pe coada Redis — din acest punct, sistemul nu mai știe și nu-i mai pasă de pe ce canal a venit mesajul.
3. Muncitorul ia plicul, află cărui magazin îi aparține, și rulează pipeline-ul (banda rulantă cu 11 stații — Diagrama 4a). Pipeline-ul vorbește cu baza de date și, dacă e nevoie, cu OpenAI.
4. Răspunsul se scrie în outbox; dispecerul îl ia de acolo și îl livrează prin clientul de canal potrivit (Meta / Telegram / SSE pe web).
5. Separat, joburile de fundal fac curățenie, calcule zilnice și programează mesajele proactive (coș abandonat, produs revenit în stoc).

Două **excepții** vizibile pe hartă, ambele intenționate: (1) pe site, `POST /web/chat` e sincron — răspunsul se întoarce direct în același request HTTP, fără outbox, pentru că pagina web așteaptă oricum răspunsul; (2) indicatorul „scrie...” ocolește outbox-ul și pleacă direct — trebuie să apară instant, nu are nicio valoare dacă ajunge cu 2 secunde întârziere.

DE REȚINUT

Sistemul e un lanț de piese **mici și specializate**, legate prin cozi și tabele — nu un program mare care face tot. Dacă o piesă cade, celelalte merg mai departe, iar mesajele așteaptă cumiți pe coadă în loc să se piardă.

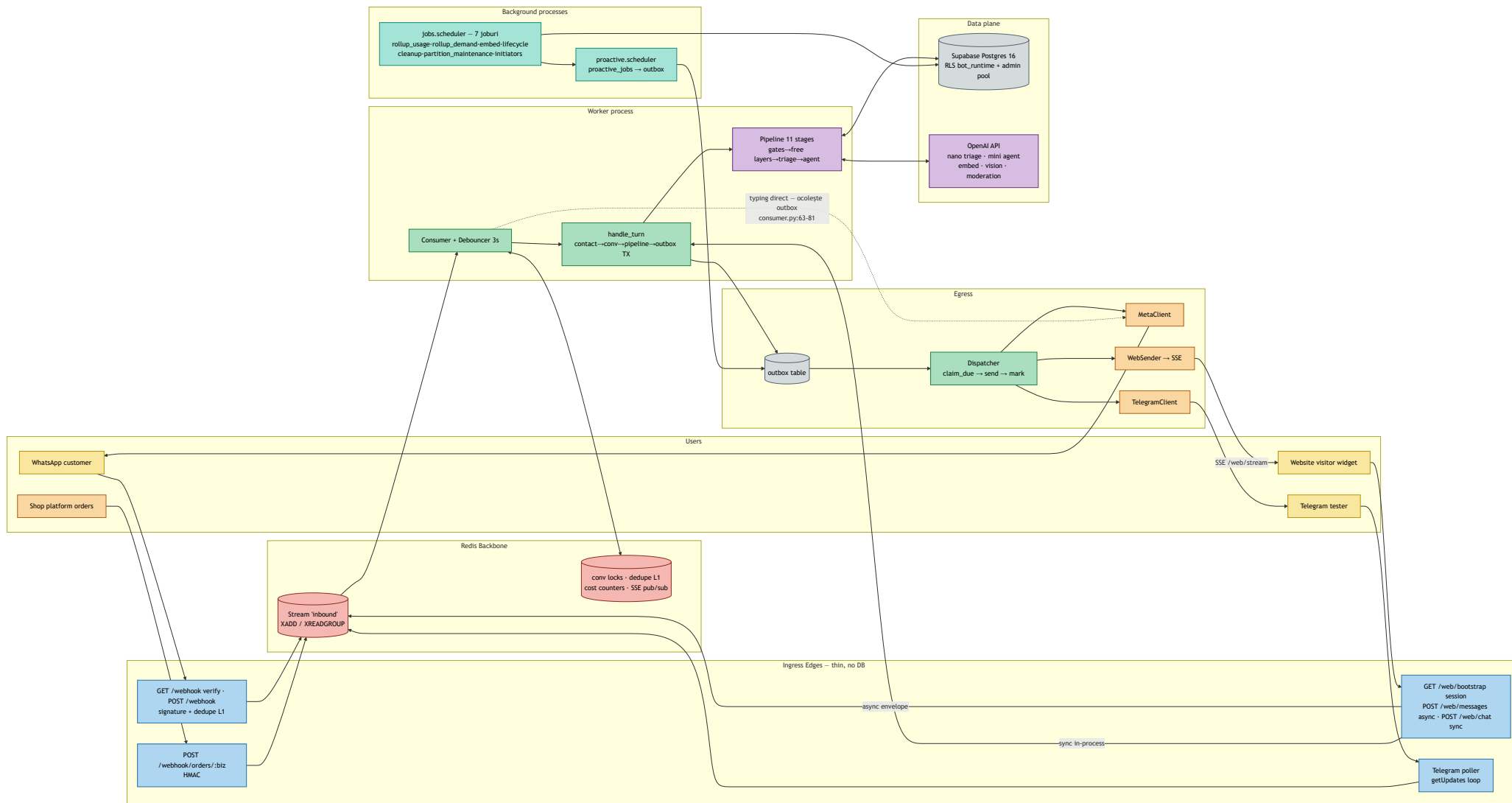


Diagrama 1 — Harta mare a sistemului. Vectorial: dă zoom pentru detalii.

Diagrama 2 — Pornirea aplicației: checklist-ul de dinainte de decolare

Ce arată

Ce verifică fiecare dintre cele șase programe (webhook, worker, dispecer, scheduler de joburi, proactiv, poller Telegram) **înainte** să se declare gata de lucru („READY”) — și în ce condiții refuză să pornească.

ANALOGIE

E checklist-ul pilotului de dinainte de decolare. Dacă un singur punct nu e în regulă, avionul **nu decolează** — nu „decolează și vedem noi în aer”. În diagramă, căsuța roșie „BOOT REFUSED — crash loud” e exact asta: refuzul zgomotos de a porni.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

Principiul e „**crapă devreme și zgomotos, nu târziu și tăcut**”. O parolă greșită de bază de date descoperită la pornire = un mesaj clar în log și un serviciu care refuză să pornească — vizibil imediat. Aceeași parolă greșită descoperită *la primul mesaj al unui client* = un client care nu primește răspuns și o oră de căutat cauza. Toate verificările scumpe s-au mutat deliberat la boot.

Logica, pas cu pas (pe muncitor, cel mai strict)

1. **Migrările sunt la zi?** Schema bazei de date e versionată. Dacă în cod există o migrare care n-a fost aplicată pe DB, muncitorul **refuză să pornească**. De ce: un cod nou care rulează pe o schemă veche produce erori subtile și corupe date — mai bine deloc decât greșit.
2. **Sunt conectat cu rolul corect?** Se verifică explicit că conexiunea de lucru folosește rolul `bot_runtime` — rolul *fără* puterea de a ocoli izolarea dintre magazine. Rol greșit = boot refuzat. Asta previne categoria cea mai gravă de bug: datele unui client ajunse la altul.
3. **Pool-urile de conexiuni se deschid „eager”** (acum, nu la nevoie): parolă greșită, DB căzut, Redis lipsă — toate crapă la boot, nu la primul mesaj.
4. Se pregătesc clienții de canal și grupul de citire de pe coadă, apoi bucla principală pornește: READY.

Procesele **opționale** ies elegant, nu crapă: proactivul fără flag-ul pornit și poller-ul Telegram fără token se opresc curat, cu mesaj în log — nu rămân procese-zombi care par vii dar nu fac nimic.

DE REȚINUT

Majoritatea „bug-urilor misterioase” din producție sunt de fapt porniri greșite. Diagrama asta arată exact ce e garantat adevărat despre un proces care a ajuns la READY: schema la zi, rolul corect, conexiunile valide. Dacă rulează, premisele sunt sănătoase.

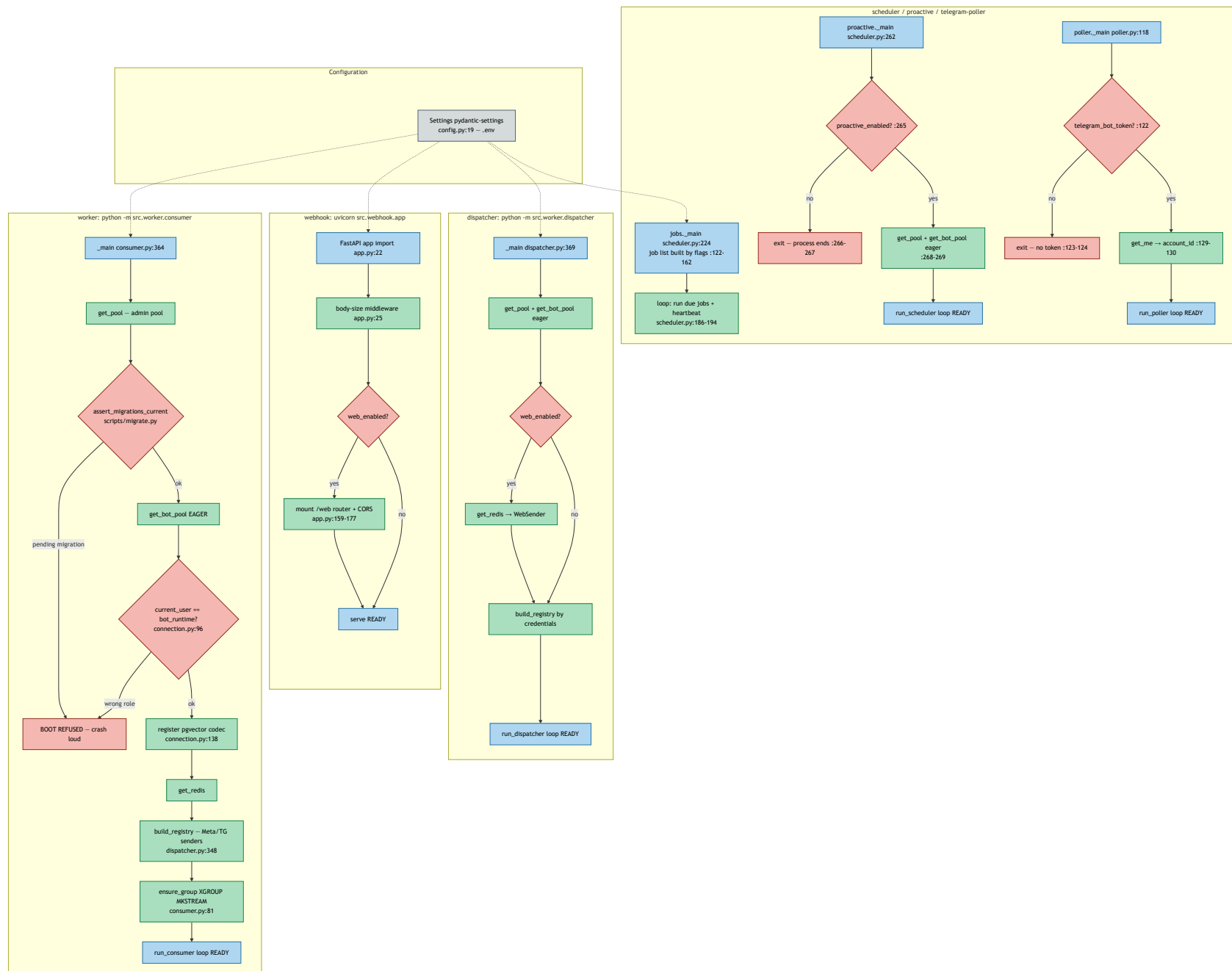


Diagrama 2 — Pornirea celor șase procese, cu porțile care pot refuza boot-ul.

Diagrama 3 — Drumul unui mesaj: de la „Send” până la răspuns

Ce arată

Povestea completă a unui singur mesaj, de la momentul în care clientul apasă „Send” pe WhatsApp până când primește răspunsul. E cea mai „narativă” diagramă din document și cea mai lungă — pentru că fiecare romb galben de pe ea e o lecție învățată: o problemă reală care s-a întâmplat sau care se întâmplă garantat la scară.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

Drumul e împărțit în **patru tronsoane cu responsabilități separate**: vama de intrare (rapidă, paranoică), muncitorul (ordonează și dozează), turul propriu-zis (gândește și salvează atomic) și livrarea (trimite și reîncearcă). Fiecare tronson poate eșua independent fără să-l dărâme pe următorul, iar mesajul nu se pierde între ele pentru că stă mereu ori pe coadă, ori în baza de date.

Logica, pas cu pas

Tronsonul 1 — vama de intrare (milisecunde):

- **E anormal de mare?** → refuzat. Apărare anti-sufocare: un mesaj de 10MB nu are ce căuta în sistem.
- **Semnătura e autentică?** Meta semnează criptografic fiecare livrare. Fără verificare, oricine ar putea să ne injecteze „mesaje de la clienți” false.
- **L-am mai văzut?** (dedupe, stratul 1) — Meta re-trimite agresiv la orice întârziere; fără asta, clientul primește răspunsuri duble.
- Mesajul intră pe coadă și confirmăm către Meta în sub 50ms. Dacă Redis e căzut, răspundem „503” — semnalul oficial prin care Meta știe să reîncerce mai târziu. Nimic pierdut.

Tronsonul 2 — muncitorul ordonează:

- „**Scrie...**” apare instant la client — feedback imediat, cost zero.
- **Debounce 3 secunde**: oamenii scriu în rafale de mesaje scurte. Așteptăm 3 secunde de liniște și lipim rafala într-un singur mesaj → un singur răspuns coerent, nu trei răspunsuri care se calcă în picioare.
- **Al cui e mesajul?** Se rezolvă canalul → magazinul (tenant-ul). Abia de acum știm în datele cui lucrăm.
- **Lacăt per conversație**: două mesaje din aceeași discuție nu se procesează niciodată în paralel (răspunsurile s-ar amesteca). Conversația e ocupată? Punem mesajul înapoi pe coadă cu o pauză.
- **Frâna de concurență per magazin**: un magazin cu trafic exploziv nu poate monopoliza toți muncitorii în detrimentul celorlalți clienți ai platformei.

Tronsonul 3 — turul propriu-zis:

- **Dedupe stratul 2**, în baza de date: plasa pentru dublurile care scapă de Redis (de exemplu după un restart). Două straturi, pentru că fiecare acoperă golul celuilalt.
- Se încarcă contextul: cine e clientul, ultimele 8 mesaje, starea discuției, faptele știute.
- **Bugetul AI al zilei e depășit?** AI-ul se stinge pentru azi, dar clientul tot primește un răspuns (degradat). Factura unui magazin nu poate exploda peste noapte.
- Rulează cele 11 stagii (Diagrama 4a) → iese un răspuns.

- **Salvarea atomică:** mesajele + răspunsul de trimis + noua stare a discuției se scriu *într-o singură tranzacție*. Ori tot, ori nimic — nu există „am salvat mesajul dar am pierdut răspunsul”.
- Abia după răspuns, în fundal: scrierea în cache, rezumatul, profilul — nimic din astea nu ține clientul în așteptare.

Tronsonul 4 — livrarea: dispecerul ia din outbox, alege forma după ce *poate* canalul (carduri pe web, carousel pe Telegram, text pe WhatsApp), trimite, reîncearcă cu pauze crescânde la eșec și, dacă epuizează încercările, marchează mesajul „mort” — vizibil în date, nu dispărut tăcut.

DE REȚINUT

Fiecare verificare de pe acest drum protejează ceva concret: dedupe = fără răspunsuri duble; debounce = un răspuns pentru o rafală; lacătul = ordinea conversației; bugetul = factura; tranzacția = coerența datelor. Nimic nu e birocrație gratuită.

Diagrama 3 — Viața unui mesaj, cap-coadă. Diagramă înaltă: dă zoom pe tronsonul care te interesează.

Diagrama 4a — Pipeline-ul: cele 11 stagii prin care trece orice mesaj

Ce arată

Banda rulantă a sistemului: cele 11 stații, în ordinea fixă în care rulează, cu toate „ieșirile devreme”. Orice stație poate produce răspunsul final — și atunci restul benzii se sare complet. Ordinea: porți de siguranță → limbă → clarificare reluată → salut → alias → cache → FAQ → triaj → handoff → agent → plasa finală.

ANALOGIE

E un filtru de apă cu straturi așezate de la **ieftin la scump**. Apa (mesajul) trece prin straturi din ce în ce mai costisitoare; dacă un strat rezolvă problema, apa iese acolo și nu mai plătim straturile scumpe.

Salutul, aliasul, cache-ul și FAQ-ul sunt gratuite; triajul costă cât un AI mic; agentul costă cât un AI mare.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

Economia dictează ordinea. Ținta: 40–60% din mesaje să nu atingă AI-ul deloc. „Bună ziua” nu are nevoie de inteligență artificială; „care e programul?” e în FAQ; „aveți șampon?” întreat a doua oară azi e în cache. Doar ce e cu adevărat nou și complex ajunge la AI-ul scump. Iar înaintea oricărei economii stau **porțile de siguranță** — pentru că niciun ban economisit nu justifică un răspuns dat unui client pe care l-a preluat deja un om, sau unui abuzator.

A doua regulă: incertitudinea = o întrebare, nu o ghiceală. Dacă triajul nu e sigur ce vrea clientul, sistemul pune o întrebare scurtă de clarificare (ieftină) în loc să lanseze AI-ul mare pe o pistă greșită (scump și enervant pentru client).

Logica, pas cu pas

1. **Porțile (cod pur):** botul e oprit pe conversația asta? a preluat un om? clientul e blocat? trimite rafale de spam? mesaj abuziv (moderare automată — prea multe abateri în 24h → blocat)? semne de risc (amenințare legală, cere explicit un om)? Tot aici: dacă a trimis o poză, AI-ul de imagini o descrie și descrierea devine text de căutare.
2. **Limba:** RO / HU / EN. Se detectează devreme pentru că *totul* de aici încolo (FAQ, cache, formulări) depinde de limbă. Un răspuns din cache în limba greșită e un bug, nu un hit.
3. **Clarificarea reluată:** dacă turul trecut i-am pus o întrebare („ce buget ai?”), răspunsul lui de acum completează formularul și reluăm de unde am rămas. A răspuns vag de prea multe ori? Nu-l mai chinuim — escaladăm la om sau intrăm direct pe vânzare.
4. **Salut?** „Bună!” primește mesajul nostru de bun venit, scris de noi — fără AI, cost zero.
5. **Alias:** fraze pe care le știm *exact* (aprobate manual în prealabil). Unele au răspunsul gata; altele ne spun doar direcția („asta e despre produse”) și sar peste triaj.
6. **Cache semantic:** am mai răspuns la o întrebare aproape identică? Servim din memorie. Excepție importantă: prețurile și stocul se *re-verifică* în catalog, nu se servesc orbește — un preț vechi din cache e o minciună.
7. **FAQ:** se potrivește cu o întrebare frecventă? Servim răspunsul pregătit.

8. **Triajul (AI-ul mic):** clasifică mesajul — vânzare / comandă / simplu / neclar. Clasificarea e *verificată*: categoria inventată de AI care nu există la noi → tratăm ca „neclar”. Întrebare cu cifre deghizată în „simplu” → o trimitem totuși la agent (AI-ul mic nu are voie să improvizeze fapte). Pentru „simplu” veritabil, AI-ul mic răspunde chiar el — mai ieftin.
9. **Handoff:** cere un om? Pe canalele cu operator, anunțăm operatorul. Pe site nu există operator — nu tăcem, ci tratăm cererea ca pe o vânzare și ajutăm singuri.
10. **Agentul (AI-ul mare):** vânzătorul propriu-zis — caută în catalog și compune recomandarea. Interiorul lui e Diagrama 4b.
11. **Plasa finală:** nimic n-a produs răspuns (AI căzut, clasificare invalidă)? Punem o întrebare de clarificare.
Niciodată tăcere — singurele tăceri permise sunt cele intenționate (om a preluat discuția / rafală de spam).

DE REȚINUT

Pipeline-ul e **liniar** — nicio stație nu sare înapoi, nu există bucle de orchestrare. Asta îl face previzibil: pentru orice mesaj poți spune exact ce stații a atins și de ce a ieșit unde a ieșit. Simplitatea asta e o alegere, nu o limitare.

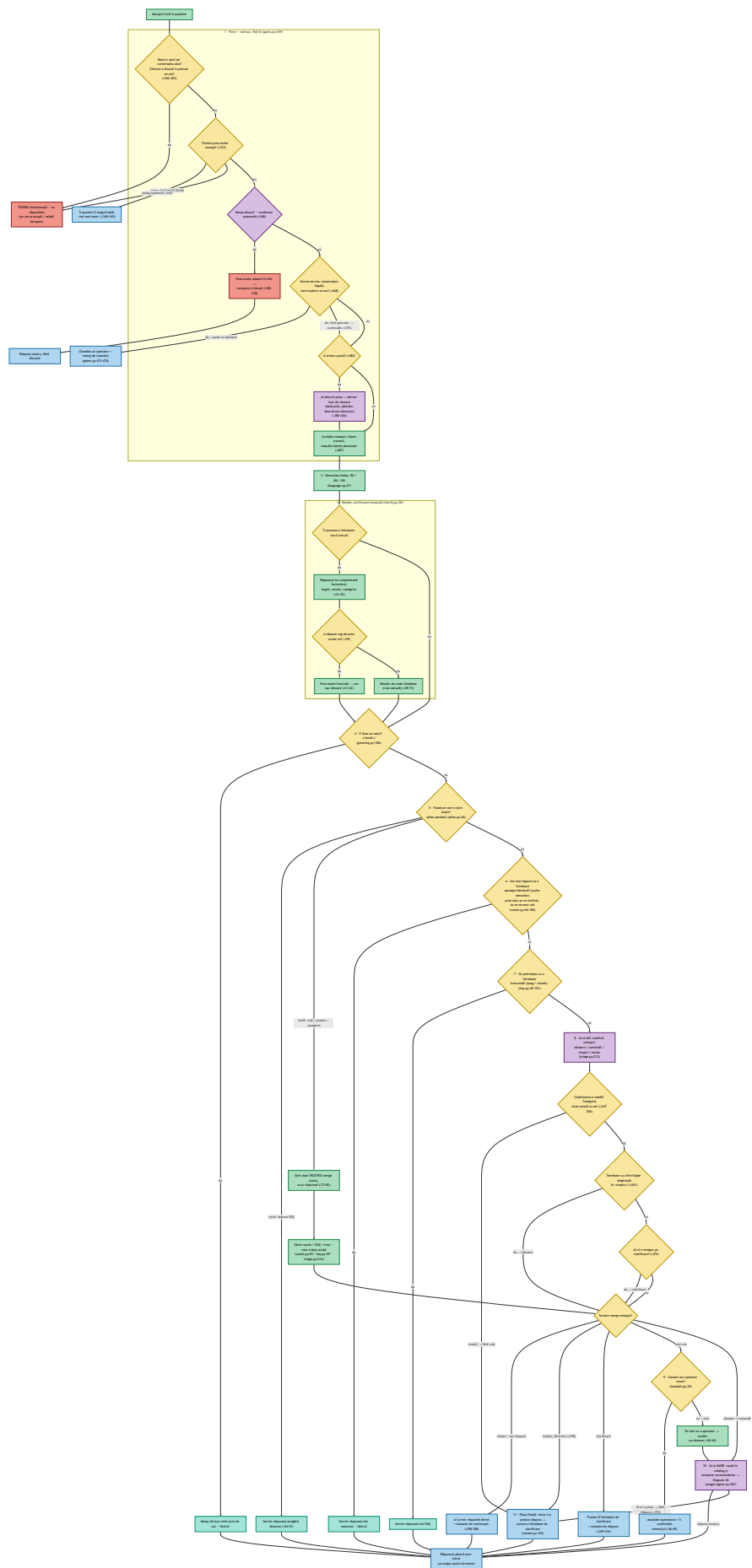


Diagrama 4a — Cele 11 stadii cu toate ieșirile devreme. Dă zoom pe stagiul care te interesează.

Diagrama 4b-1 — Creierul vânzător, privirea de sus: fazele A–F

Ce arată

Interiorul stagiului 10 (agentul — AI-ul mare) văzut de la înălțime: șase faze, A→F. Observația cheie pe care diagrama o face vizibilă: **din șase faze, patru nu cheamă niciun AI**. AI-ul e „angajat” pe o poziție precisă — să caute și să formuleze — iar codul păstrează controlul asupra a ce e adevărat.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

Invariantul central al întregului sistem: modelul AI nu are ultimul cuvânt pe cifre și linkuri. AI-ul propune; codul dispune. Orice preț, link sau număr care ajunge la client trebuie să existe în datele reale extrase din catalog în acest tur. De aceea fazele E (decizia) și F (verificarea) sunt cod determinist — ele sunt „redacția” care verifică articolul înainte de publicare.

Logica, pas cu pas

- **A · Pregătire + siguranță.** Primul lucru, înaintea oricărui alt pas: dacă clienta a declarat ceva relevant pentru siguranță (de exemplu că e însărcinată), informația se *persistă* în starea conversației ACUM. De ce atâta grabă: istoricul vizibil are doar 8 mesaje — fără persistare, o declarație de la turul 9 ar „dispărea” și un produs contraindicat ar putea reapărea în recomandări. Nu e un risc acceptabil.
- **B · Scurtături fără AI.** „Dă-mi linkul”, „compară-le”, „detalii”, „ce zic recenziile” — despre produse *deja afișate*. Toate au răspuns determinist: știm exact la ce produse se referă, nu e nimic de „înțeles”. Răspuns imediat, cost zero.
- **C · Construirea instrucțiunilor.** „Fișa postului” AI-ului se generează din baza de date a magazinului (categorii, vocabular) plus tot ce știm din discuție (buget, restricții, ce i-am arătat deja). Nimic hardcodat — același cod servește un magazin de cosmetice și unul de piese auto.
- **D · Bucla de unelte.** AI-ul mare primește unelte controlate (caută în catalog, adu detalii, compară) și are voie la **maxim 3 apeluri** — limită impusă în cod, nu rugămintă în prompt. „Mai arată-mi” nici nu intră în buclă: pagina următoare din rezultatele deja găsite se servește determinist.
- **E · Codul decide.** Ce se arată efectiv clientului decide codul, printr-o listă de precedentă (Diagrama 4b-2).
- **F · Compunerea și verificarea.** Se alege forma răspunsului și se verifică fiecare cifră (Diagrama 4b-3).

DE REȚINUT

Agentul nu e „un AI care face tot” — e un sandwich: cod → AI → cod. AI-ul stă la mijloc, înconjurat de pregătire deterministă înainte și de verificare deterministă după.

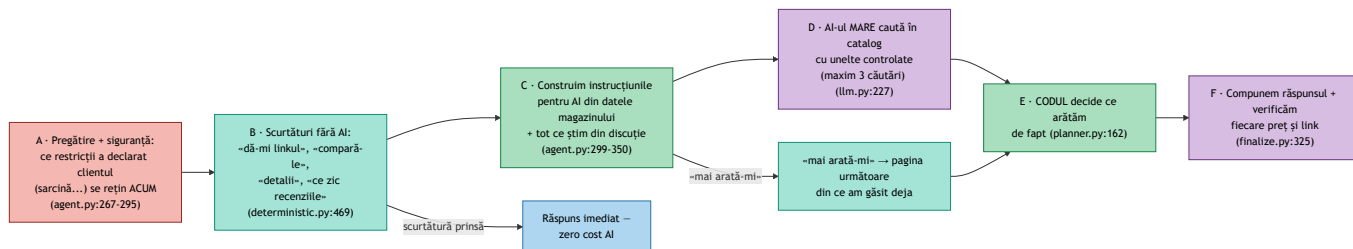


Diagrama 4b-1 — Cele șase faze ale agentului. Patru din șase nu cheamă niciun model AI.

Diagrama 4b-2 — Faza E: lista de precedență (codul decide ce arătăm)

Ce arată

Cele șase cazuri speciale pe care codul le verifică, **în ordine**, după ce AI-ul și-a terminat căutările — plus filtrul final de siguranță prin care trece totul. Primul caz care se potrivește câștigă.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

De ce listă de precedență și nu arbore de decizie? Un arbore cu 6 niveluri de ramificare devine imposibil de citit și de extins — fiecare caz nou dublează combinațiile. O listă se citește de sus în jos, se extinde adăugând un rând, și nu poate „încurca” ramurile. Când regulile de business se schimbă (și se schimbă), simplitatea asta e diferența dintre o modificare de 10 minute și un bug de producție.

Logica, pas cu pas — cele șase cazuri, cu exemple

1. **Întreabă de comanda lui, dar nu e logat pe site?** → Îi cerem să se logheze. Fără cont verificat nu putem căuta comenzi — altfel oricine ar putea întreba de comenzile oricui. E o regulă de securitate, nu de comoditate.
2. **Vrea să cumpere, dar AI-ul a „uitat” să-i facă linkul de plată?** → Codul creează el linkul, pe același drum contabilizat. O vânzare nu are voie să se piardă din cauza unei scăpări a modelului — banii clientului nostru (magazinul) sunt pe masă.
3. **Tocmai a pus ceva în coș?** → Îi arătăm produse care *merg împreună* cu ce a luat (cross-sell), trecute prin filtrul de siguranță. A mers → gata; n-a mers → continuăm lista.
4. **Întreabă „care dintre ELE e cea mai hidratantă?”** (superlativ pe produsele deja arătate) → Recitim produsele din catalog și răspundem *pe fapte*, nu din memoria AI-ului. Memoria modelului e nesigură; catalogul nu minte.
5. **Cere „ceva mai ieftin”?** → Căutăm STRICT mai ieftin decât ce a văzut. Nu există? Îi spunem cinstit — și **notăm golul de preț** pentru comerciant: „clienții cer varianta sub 50 lei și n-o avem” e informație de bani, nu un eșec de bot. Faptul se scrie exact aici, la sursă, pentru că mai târziu n-ai cum să distingi „n-am găsit nimic” de „n-am găsit nimic *mai ieftin*”.
6. **AI-ul n-a adus produse, dar clientul vorbește despre cele deja arătate?** → Le recitim din catalog („rehidratare”). E plasa care evită un „n-am găsit nimic” absurd când clientul întreabă de produsul pe care i l-am arătat acum 30 de secunde.

Filtrul final de siguranță

Orice ar fi adus căile de mai sus, **orice produs contraindicat** (de exemplu retinoizi pentru o clientă însărcinată) **e scos aici**, la capătul pâlniei. De ce încă un filtru, dacă fiecare cale are deja filtrul ei? Pentru drumurile *viitoare*: dacă cineva adaugă la anul o cale nouă și uită filtrul, plasa finală tot prinde produsul. Pe seturile deja filtrate nu costă nimic (nu schimbă nimic); pe o scăpare, salvează tot. Setul care iese de aici devine **singura sursă de adevăr** pentru răspuns, carduri și memoria discuției.

DE REȚINUT

Faza E e locul unde se vede filozofia „AI-ul propune, codul dispune”: toate cazurile cu miză (bani,

securitate, siguranță) sunt tratate de cod, pe fapte recitite din catalog — niciodată din memoria modelului.

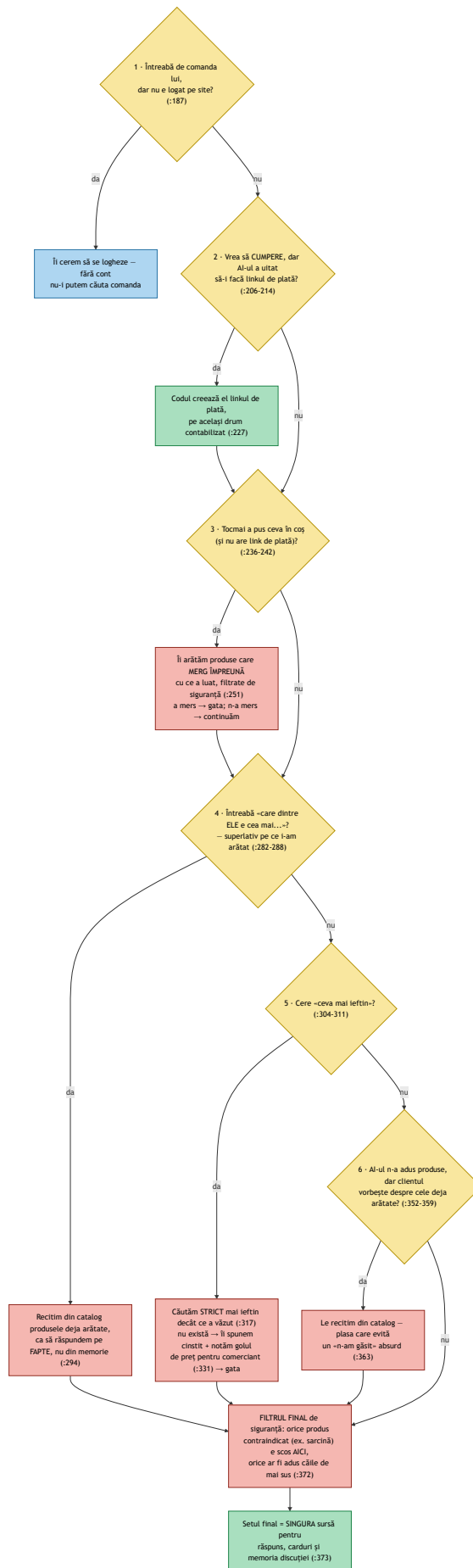


Diagrama 4b-2 — Faza E: cele șase cazuri în ordine + filtrul final de siguranță.

Diagrama 4b-3 — Faza F: cum se alege forma răspunsului

Ce arată

Ultima fază a agentului: în ce *formă* pleacă răspunsul (tabel comparativ, carduri de produs, text simplu, răspuns despre comandă, sau „nu am găsit”) — și, esențial, treptele de cădere controlată când o formă nu iese bine.

ANALOGIE

E ca aterizarea unui avion: planul A e pista principală (cardurile frumoase), planul B e pista de rezervă (text simplu, verificat), planul C e aterizarea de urgență (formulare scrisă de cod, fără nicio cifră). La fiecare treaptă pierzi din confort, dar **niciodată nu te prăbușești** — și niciodată nu minți pasagerii.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

Degradare grațioasă în trepte: calitatea prezentării poate scădea controlat, dar corectitudinea nu se negociază la nicio treaptă. Un răspuns mai urât dar adevărat bate întotdeauna un răspuns frumos cu un preț inventat. Fiecare cădere de pe o treaptă pe alta se înregistrează în telemetrie — dacă „rich” cade prea des, vedem în date, nu ghicim.

Logica, pas cu pas

1. **A cerut o comparație?** → Tabelul comparativ e construit integral de **cod**: AI-ul nu scrie nicio cifră în celule. Diferențele dintre produse sunt făcute din catalog, nu proză de AI.
2. **Avem produse de arătat (vânzare)?** → AI-ul compune recomandarea structurată (cardurile — Diagrama 4c arată cât de strâns e controlat). A ieșit ceva valid? → Trimitem cardurile + butonul de plată. N-a ieșit? → **cădem pe text simplu**, iar motivul căderii se înregistrează.
3. **Textul trece prin validator:** fiecare preț, link, cifră, afirmație de stoc sau de sănătate e verificată contra datelor reale extrase în acest tur. Invalid? → **o singură reîncercare**, cu explicația exactă a greșelii („prețul 45 lei nu există în date”). Tot invalid? → **formulare scrisă de cod, fără cifre** — plictisitoare, dar imposibil de greșit.
4. **Era despre o comandă?** → Răspunsul e legat STRICT de datele comenzii lui (status, AWB) — nimic general, nimic inventat.
5. **„Nu am găsit”:** mesaj sigur + butoane de continuare. Detaliu crucial: acest răspuns **NU se salvează în cache**. De ce: dacă „n-am găsit” ajunge în cache, orice client cu o întrebare similară primește „n-am găsit” la infinit, chiar și după ce produsul apare în stoc. S-a întâmplat pe viu — de-atunci e regulă.

DE REȚINUT

Formă flexibilă, adevăr rigid. Sistemul preferă să-ți dea mai puțin (text în loc de carduri, formulare fără cifre) decât să-ți dea ceva fals. Iar fiecare compromis e măsurat, ca să-l putem repara, nu doar tolera.

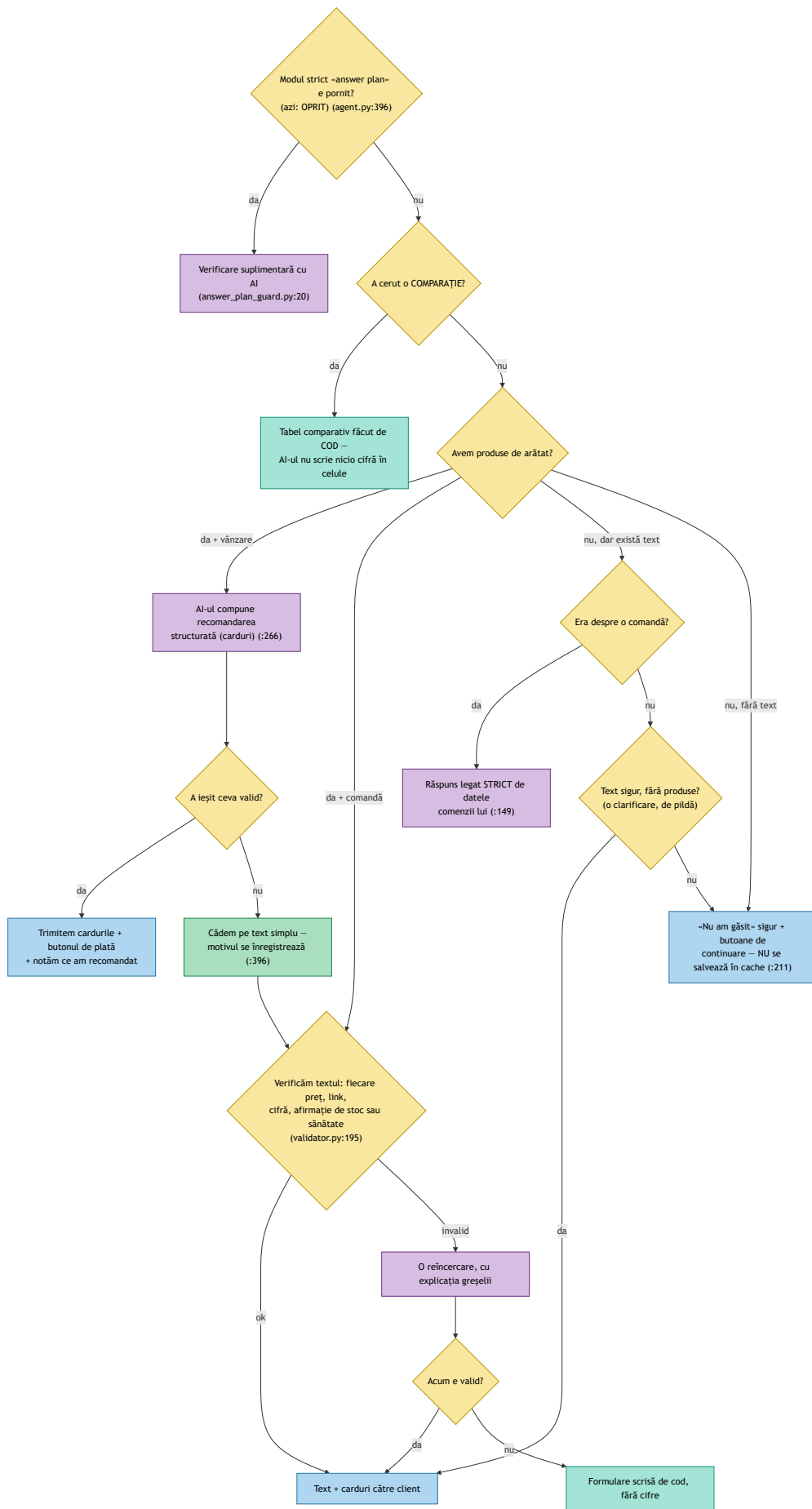


Diagrama 4b-3 — Faza F: alegerea formei + treptele de degradare controlată.

Diagrama 4c — Cardurile de produs: cum ancorăm AI-ul în fapte

Ce arată

Fabrica de carduri: drumul de la „AI-ul vrea să recomande produse” până la cardurile finale pe care le vede clientul — cu toate punctele de control. E echivalentul validatorului, dar pentru calea „bogată” (cardurile), care servește majoritatea recomandărilor.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

Problema: un AI lăsat să scrie liber despre produse va inventa, mai devreme sau mai târziu, un preț, un procent de reducere sau o promisiune („elimină ridurile în 7 zile”). **Soluția:** AI-ul nu completează cardul. AI-ul emite DOAR cuvintele de legătură + *referințe* către produse (ID-uri). Codul „hidratează” apoi faptele — preț, poză, link, stoc — direct din catalog. AI-ul fizic *nu poate* pune un preț greșit pe card, pentru că nu el pune prețul.

Logica, pas cu pas

1. **Pregătirea:** codul calculează pe ce axe chiar diferă produsele din set (preț, mărime, proprietăți) — ca textul AI-ului să vorbească despre diferențe REALE, nu despre platitudini.
2. **AI-ul emite JSON structurat:** introducere, motivarea potrivirii, educație scurtă — plus ID-urile produselor la care se referă. Doar cuvinte și referințe.
3. **Controlul de apartenență:** fiecare element trebuie să refere un produs care CHIAR a fost extras din catalog în acest tur. Referință inventată → elementul se aruncă întreg. AI-ul nu poate „strecura” un produs din imaginația lui.
4. **Curățenia câmp cu câmp (scrub):** în textele AI-ului, cifrele, procentele și afirmațiile neverificabile se șterg câmp cu câmp — un câmp suspect devine gol, restul cardului trăiește.
5. **Claim-uri medicale** → **câmpul se ARUNCĂ**, nu se ajustează. „Tratează eczema”, „sigur în sarcină”, „recomandat de medici” — niciuna nu are voie să apară, indiferent cât de convingătoare e formularea. Nu e o preferință de stil: e răspundere juridică.
6. **Badge-urile** („reducere”, „top vânzări”) vin doar din semnale reale din date, cu praguri configurate — nu din entuziasmul AI-ului.
7. **Alegerea „recomandării principale”** e deterministă (cod, pe criterii), și se suprimă când produsele nu sunt din categoria cerută.
8. **Aplatizarea:** pentru canalele care nu pot afișa carduri (de exemplu WhatsApp text), cardul se transformă în text simplu — aceleași fapte, altă haină.

DE REȚINUT

Regula de aur a căii bogate: **AI-ul scrie doar cuvinte; faptele vin din catalog.** Un card poate ieși mai sec (câmpuri curățate), dar nu poate ieși mincinos — minciuna e imposibilă prin construcție, nu prin bune intenții.

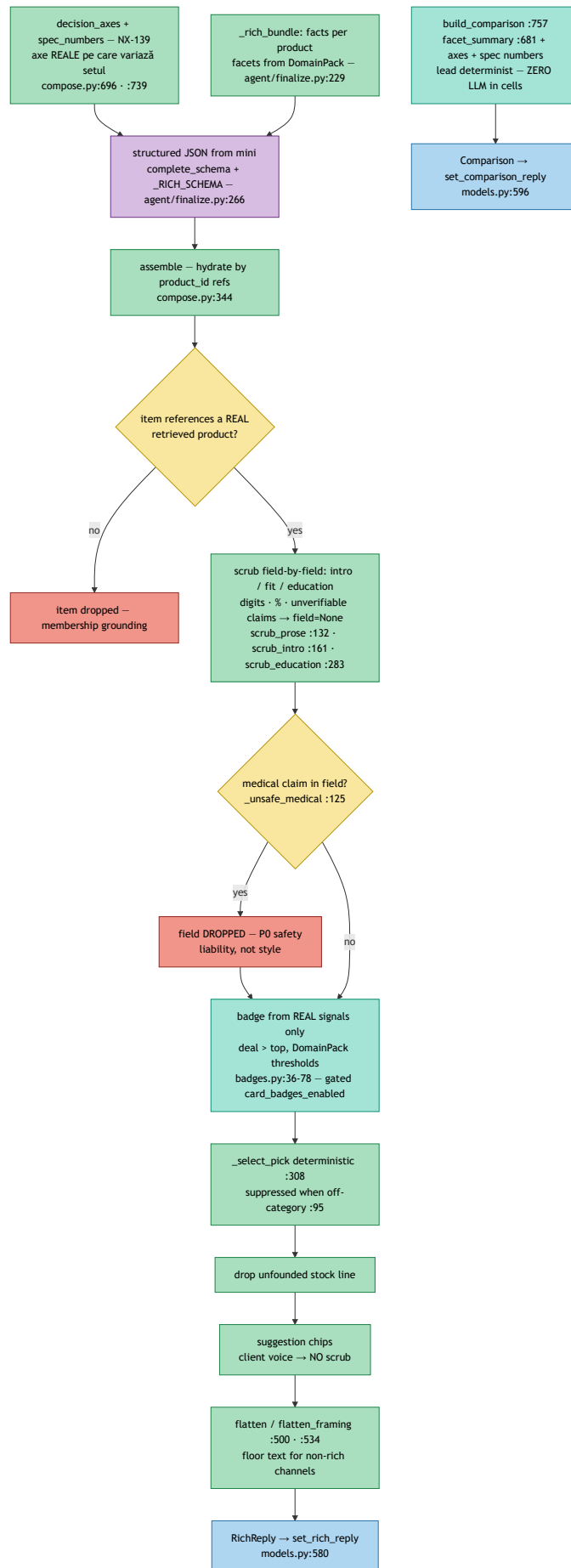


Diagrama 4c — Lanțul de ancorare în fapte al cardurilor de produs.

Diagrama 5 — Căutarea în catalog: cum găsim produsul potrivit

Ce arată

Ce se întâmplă când AI-ul cere o căutare în catalog: traducerea limbajului clientului, căutarea dublă (pe cuvinte + pe înțeles), planul de rezervă când nu iese nimic, filtrul de siguranță, diversificarea primei pagini și — foarte important pentru business — înregistrarea a ceea ce s-a căutat și NU s-a găsit.

ANALOGIE

E un vânzător bun într-un magazin fizic: **traduce** ce spui („am tenul gras”) în limbajul rafturilor (categoria „oily”); **ține minte pe ce raft sunteți** (dacă scrii „mai ifetin” cu typo, nu te duce brusc la alt raion); caută și după **etichete** și după **sens**; iar dacă nu găsește exact ce ceri, **slăbește pretențiile în ordine** — dar nu-ți vinde niciodată alt brand pretinzând că e cel cerut.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

- 1. Căutare hibridă:** căutarea pe cuvinte prinde potriviri exacte („niacinamidă”), dar ratează sinonime; căutarea pe înțeles prinde „ceva pentru pori dilatați” fără cuvinte comune, dar poate aluneca tematic. Combinate într-un singur clasament, fiecare acoperă golul celeilalte.
- 2. Scara de rezervă e ordonată deliberat:** criteriile secundare se slăbesc pe rând, într-o ordine scrisă — iar **brandul cerut nu se slăbește niciodată**. Dacă clientul cere brandul X și nu-l avem, îi spunem cinstit „nu lucrăm cu X” — nu-i prezentăm brandul Y ca și cum ar fi ce a cerut. Încrederea nu se răscumpără.
- 3. Golul de cerere e aur:** tot ce s-a căutat și nu s-a găsit (produs lipsă, variantă lipsă, nimic găsit) se notează pentru comerciant. „10 clienți pe săptămână cer serul pe care nu-l ai în stoc” e cea mai valoroasă informație pe care botul o poate produce — bani lăsați pe masă, măsurați exact.

Logica, pas cu pas

- 1. Traducerea:** vorbele clientului devin chei de catalog („ten gras” → **oily**; „cu niacinamidă” → filtru pe ingredient).
- 2. Moștenirea raftului:** căutare nouă în aceeași discuție, fără categorie nouă → păstrăm categoria curentă. Un typo nu ne mai aruncă pe alt raion.
- 3. Aceeași căutare ca data trecută?** → servim pagina următoare din rezultatele deja găsite. Fără AI, fără cost, fără așteptare.
- 4. Căutarea dublă:** fraza devine un vector de înțeles (un singur apel de embedding), apoi rulează căutarea pe cuvinte + căutarea pe înțeles, iar listele se combină într-un singur clasament.
- 5. Nimic găsit?** → slăbim următorul criteriu secundar din plan și reîncercăm. Planul epuizat? → spunem cinstit „nu am găsit” (și acest răspuns NU se salvează în cache).
- 6. Filtrul de siguranță pe TOT setul, ÎNAINTE de memorare.** Poziția e esențială: dacă am filtra doar prima pagină, un produs contraindicat ar rămâne în rezerva de rezultate și ar reapărea la „mai arată-mi altele”. Filtrat înainte de memorare = nu mai există nicăieri în tur.

7. **Diversificarea:** prima pagină acoperă ieftin / mediu / scump și branduri diferite — șase opțiuni reale, nu șase clone ale aceluiași produs.
8. **Prima pagină:** maxim 6 produse, fără cele deja arătate clientului.
9. **Telemetria cererii + memoria sesiunii:** notăm golurile de catalog și ținem minte restul rezultatelor pentru „mai arată-mi”.
10. **Onestitate pe brand și pe produse numite:** brand absent → „nu lucrăm cu X”; produs numit care nu există → spunem clar, dar oferim alternativele (și golul tot se înregistrează).

Degradare: dacă serviciul de embeddings pică, căutarea continuă doar pe cuvinte — mai slabă, dar funcțională. Sistemul nu se oprește pentru că o componentă e răcită.

DE REȚINUT

Căutarea nu e doar „găsește produse” — e și **instrumentul de măsură al cererii**. Fiecare căutare eșuată e o dată de business înregistrată la sursă, nu un eșec aruncat la gunoi.

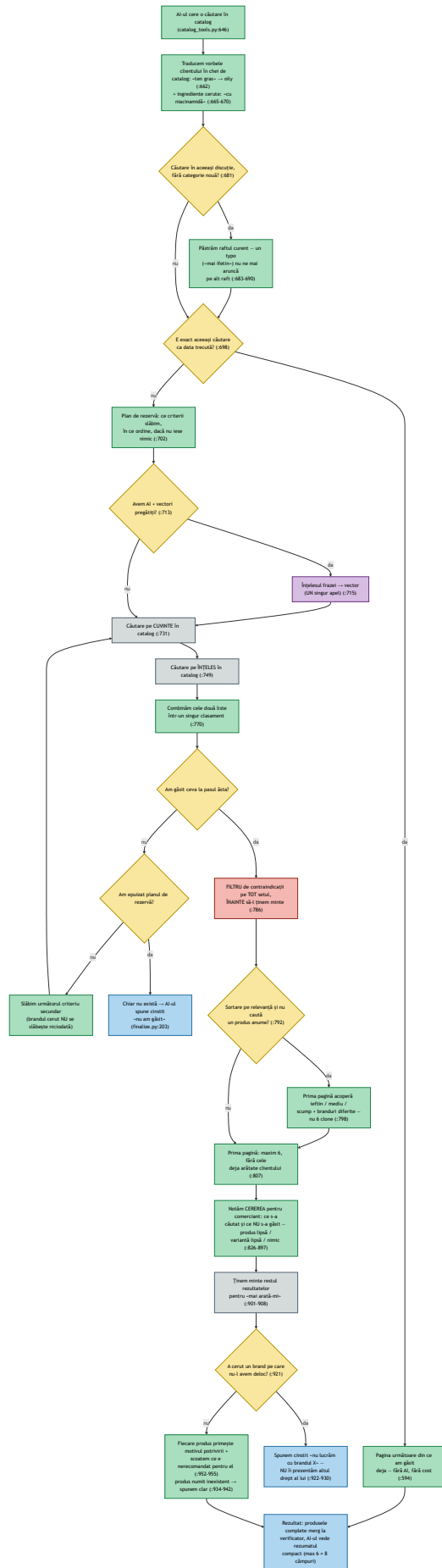


Diagrama 5 — Căutarea în catalog, cu scara de rezervă și filtrul de siguranță.

Diagrama 6 — Memoria conversației: ce ține minte botul și cum

Ce arată

Cele patru momente ale memoriei într-un tur: ce se **încarcă** la început, ce **vede** efectiv AI-ul, ce se **modifică** în timpul turului și ce se **salvează** la final — plus ce se întâmplă în fundal, după ce clientul și-a primit deja răspunsul.

ANALOGIE

Un vânzător cu carnețel: nu ține minte tot ce-ai vorbit vreodată (imposibil și scump), ci **ultimele schimburi** (8 mesaje), **o fișă scurtă despre tine** (buget, brand preferat, restricții — faptele stabile), **un rezumat** al discuțiilor lungi și **lista cu ce ți-a arătat deja**. Carnețelul are dimensiune fixă — dacă nu mai încape, se rescrie esențialul, nu se cumpără carnețel mai mare.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

- Bugete impuse în COD, nu în prompturi:** istoric maxim 8 mesaje, stare maximă 8KB, transcript compact. Contextul trimis la AI costă bani la fiecare tur — iar „disciplina prin rugăminți în prompt” nu ține niciodată; limita din cod ține întotdeauna.
- Stare = referințe, nu obiecte:** în „ce i-am arătat” ținem doar {id, nume, preț} — nu obiecte întregi de produs. De aceea orice cale care judecă produse le *recitește din catalog* întâi: o referință nu-ți spune compoziția produsului, iar prețul de acum 3 zile poate fi altul azi.
- Faptele stabile trăiesc separat de istoric:** bugetul, brandul preferat, restricțiile declarate se extrag și se persistă — pentru că istoricul de 8 mesaje le-ar „uita” natural după câteva tururi.

Logica, pas cu pas

- La început de tur** se încarcă: ultimele 8 mesaje, starea discuției (max 8KB: ce i-am arătat, ce caută, coșul), rezumatul (dacă discuția e lungă), profilul clientului și faptele stabile despre el.
- AI-ul vede** o versiune compactă: transcriptul redus (max 6 tururi / 1200 caractere) + blocuri ordonate de profil, fapte și stare. Doar faptele marcate ca sigure de injectat ajung în prompt — datele personale și cele medicale sunt filtrate la sursă.
- În timpul turului**, agentul actualizează ce caută clientul, iar uneltele (de exemplu coșul) își cer modificările lor.
- La salvare**, totul intră într-o singură tranzacție, cu un „lacăt optimist”: dacă alt proces a apucat să salveze între timp, nu-i suprascriem munca orbește. Modificările cerute de unelte câștigă ultimele (o scriere reală în coș bate o intenție).
- După răspuns, în fundal** (clientul nu așteaptă nimic din astea): răspunsul se salvează în cache-ul semantic dacă merită (zile pentru informații statice, minute pentru preț/stoc); discuțiile lungi primesc rezumat de la AI-ul mic; și tot AI-ul mic extrage fapte noi despre client + actualizează scorul de interes.

DE REȚINUT

Memoria e **stratificată după durata de viață**: mesajele recente (volatile), starea turului (scurtă), faptele

despre client (persistente), rezumatul (comprimat). Fiecare strat are buget fix impus în cod — de aceea costul unui tur e previzibil indiferent cât de lungă e conversația.

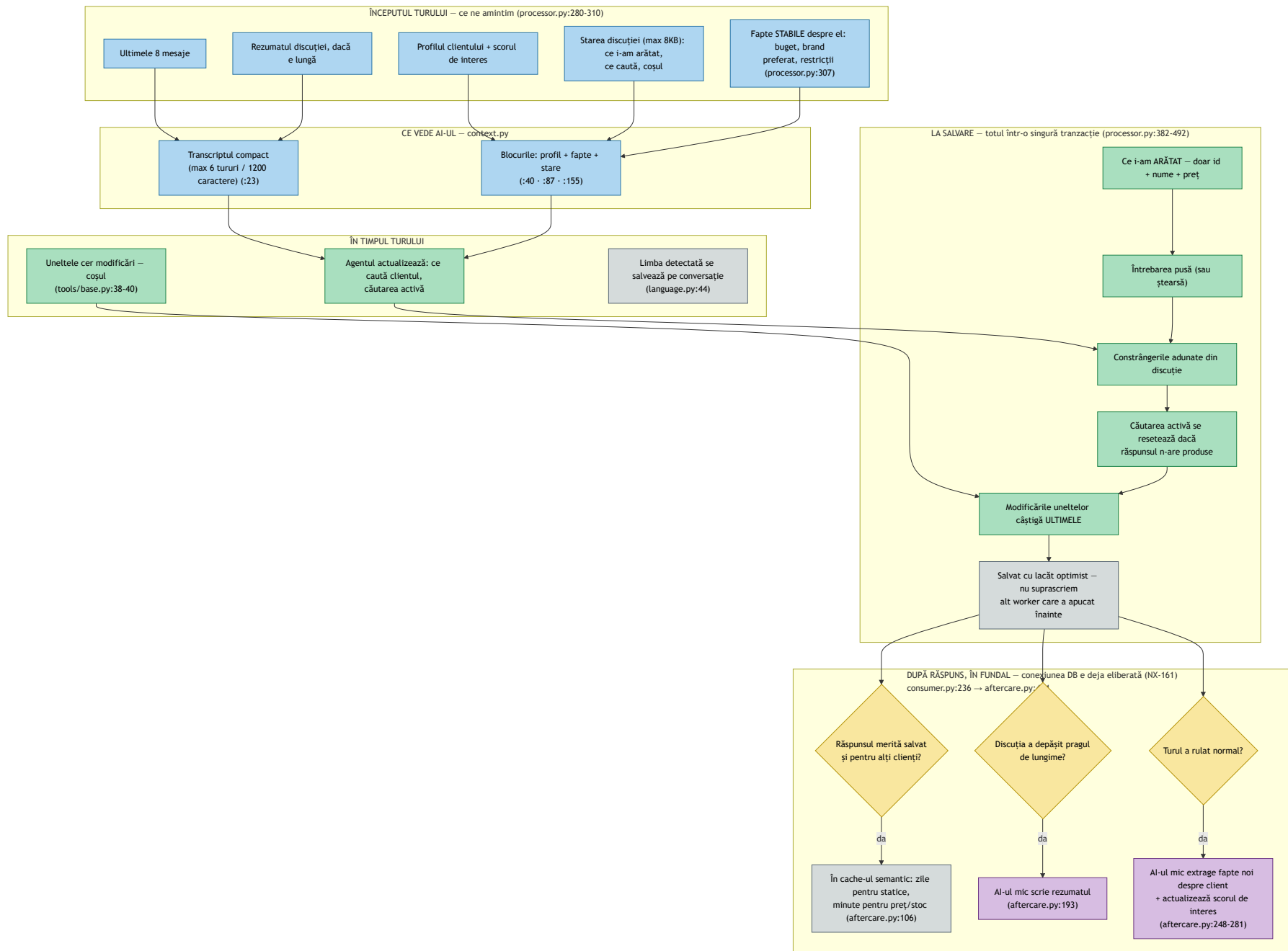


Diagrama 6 — Memoria conversației: încărcare → folosire → modificare → salvare → fundal.

Diagrama 7 — Baza de date: cum garantăm că datele unui magazin nu ajung la altul

Ce arată

Cum vorbește sistemul cu baza de date: cele două „uși” (pool-uri de conexiuni) cu puteri diferite, ritualul de la fiecare împrumut de conexiune, și polițistul din baza de date (RLS) care filtrează fiecare interogare.

ANALOGIE

Un hotel cu carduri de acces: muncitorul primește un card care deschide **doar camera clientului curent** — nu master-key. La fiecare împrumut de card, recepția verifică: „ești cine spui că ești și mergi în camera corectă?” Dacă nu → alarmă, nu „mergi oricum”. Master-key-ul (conexiunea privilegiată) există, dar e folosit doar la recepție, pentru un singur lucru: să afle *în ce cameră* trebuie să meargă cardul.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

Izolarea multi-tenant e apărată în trei straturi independente:

Stratul 1 — codul: fiecare interogare are filtrul de magazin (`WHERE business_id = ...`) scris explicit. Asta e apărarea primară.

Stratul 2 — rolul: muncitorul se conectează cu un rol care NU are puterea de a ocoli regulile de izolare, iar la fiecare împrumut de conexiune se setează „eticheta” magazinului curent și se *verifică* potrivirea (nepotrivire = eroare CRITICĂ, oprire).

Stratul 3 — RLS, polițistul din DB: reguli în baza de date care filtrează automat orice interogare pe eticheta setată. Efectul plasei: un query cu bug întoarce **0 rânduri**, nu datele altui client. Bug-ul devine vizibil și inofensiv, în loc de scandal de confidențialitate.

Logica, pas cu pas

- **Ușa privilegiată** (admin) e folosită pentru exact două lucruri: lookup-ul „canalul acesta → care magazin?” (rulează ÎNAINTE să știm tenant-ul — e chiar operația care îl află; e *singura* excepție, documentată) și joburile de mentenanță fără date personale. **Orice alt query pe ușa privilegiată e, prin definiție, un bug de izolare** — și există un test automat în CI care caută exact asta.
- **Ușa de lucru** (bot): la fiecare împrumut se setează eticheta magazinului → se verifică potrivirea rol + etichetă → toate citirile și scrierile turului trec prin filtrul RLS → la returnarea conexiunii, eticheta se șterge (conexiunea următoare nu moștenește nimic).
- **Scrierea de la finalul turului e atomică:** mesaje + outbox + stare + marcajul de dedupe, într-o singură tranzacție. Scrierile „best-effort” (analytics, cache, rezumat) au fiecare plasa lor: un eșec acolo se loghează, turul continuă.

DE REȚINUT

Nu ne bazăm pe „codul e scris corect” — ne bazăm pe **straturi care se verifică reciproc**. Codul filtrează, rolul verifică, RLS-ul plasează. Ca să scurgi date între magazine ar trebui să greșești în trei locuri independente simultan.

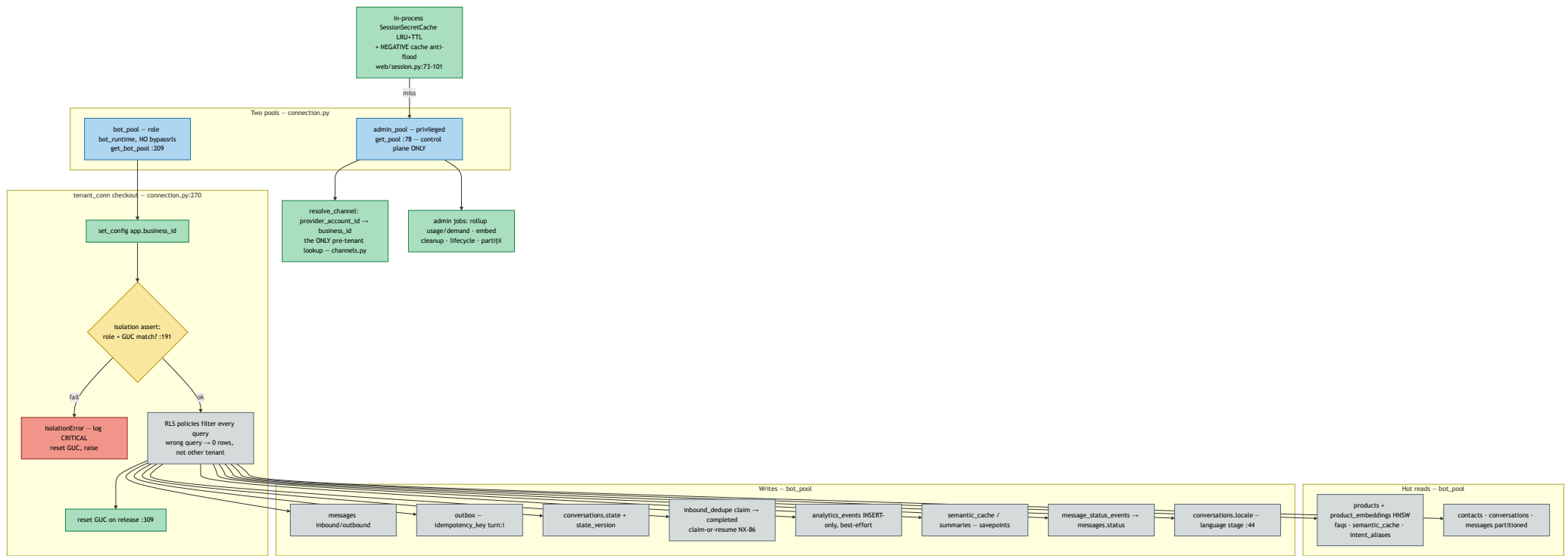


Diagrama 7 — Cele două pool-uri, ritualul de izolare și plasa RLS.

Diagrama 8 — Serviciile externe: toate „porturile” spre lume

Ce arată

Inventarul complet al dependențelor externe și punctul exact prin care vorbim cu fiecare: OpenAI (AI-ul), Meta (WhatsApp), Telegram, widgetul de pe site, platforma de comenzi a magazinului, operatorul uman și Redis. O singură pagină care răspunde la întrebarea „dacă X cade, pe unde ne doare?”.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

O singură „cusătură” per serviciu extern. Toate apelurile către OpenAI — triaj, agent, embeddings, moderare, descriere de imagini — trec printr-un singur modul, cu politica de reîncercare într-un singur loc (reîncercare mărginită, care respectă indicațiile de așteptare ale serverului). Când OpenAI schimbă ceva sau are o pană, reparăm într-UN loc, nu vânam apeluri împrăștiate prin tot codul. Aceeași regulă pentru Meta, Telegram și web: fiecare canal are exact un client de trimitere, înregistrat în registrul de canale.

Logica — ce intră și ce iese

- **Intrări:** webhook-ul semnat de la Meta, long-polling-ul de la Telegram, mesajele widgetului web (cu sesiune semnată HMAC), webhook-ul de comenzi de la platforma magazinului (semnat și el) și descărcarea media de la Meta (pozele clienților).
- **Ieșiri:** trimerile prin clientul Meta / Telegram / web-SSE (cu reluarea mesajelor pierdute dacă browserul s-a deconectat scurt) și notificarea operatorului uman la handoff — trimisă „best-effort” și **fără date personale** în conținut.
- **OpenAI e marcat distinct** (mov) pentru că e singura dependență care „gândește”: restul transportă. Diagrama arată și cele cinci roluri separate: AI mic (triaj, rezumat, profil), AI mare (agent, compunere), embeddings, moderare (gratuită), vision.
- **Redis** apare ca serviciu de sine stătător: cozi, lacăte, dedupe, contoare de cost, canalul SSE — toate trec prin el.

DE REȚINUT

Diagrama asta e „harta de risc extern”: pentru fiecare serviciu de pe ea există un plan de degradare (detaliat în Diagrama 9). Niciun serviciu extern căzut nu are voie să însemne „sistemul tace”.

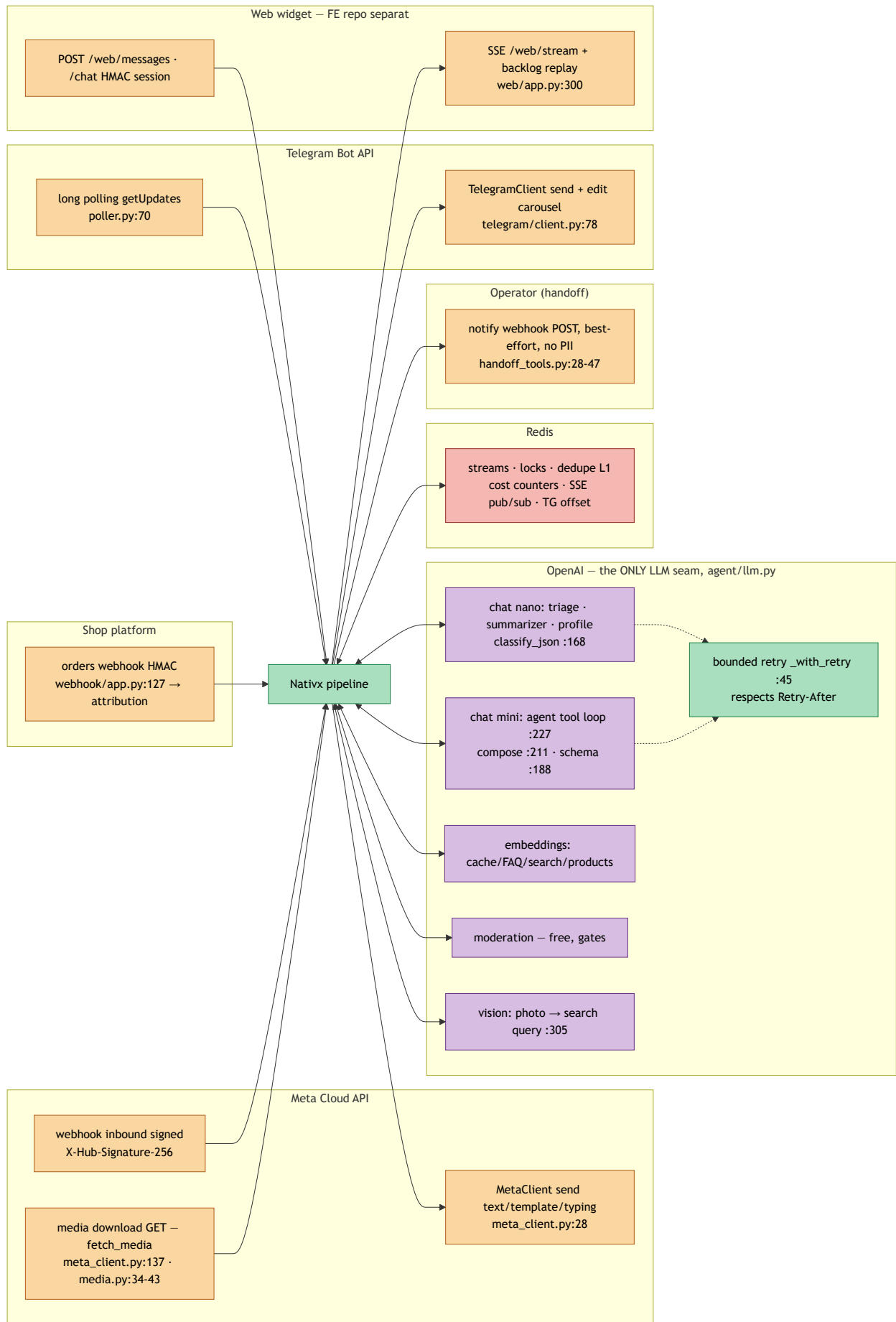


Diagrama 8 — Toate dependențele externe și cusăturile prin care vorbim cu ele.

Diagrama 9 — Erorile: ce se întâmplă când ceva pică

Ce arată

Catalogul complet al eșecurilor posibile, grupate pe familii — AI-ul pică, validatorul respinge, bugetul se termină, infrastructura se clatină, livrarea eșuează — și, pentru fiecare, planul scris dinainte: ce se degradează, ce se reia, ce se pierde și ce rămâne vizibil.

ANALOGIE

E manualul de urgențe al unui spital: nu speră nimeni că nu vor fi urgențe — se știe dinainte cine ce face la fiecare tip de urgență. Eșecul nu e tratat ca excepție rușinoasă, ci ca stare normală de funcționare la scară: la mii de mesaje pe zi, „rar” înseamnă „zilnic”.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

Regula supremă: **niciodată tăcere**. Lanțul de degradare e explicit și fiecare treaptă e cod, nu speranță: cardurile pică → text simplu; textul e invalid → o reîncercare → formulare fără cifre; AI-ul e mort → întrebare de clarificare; bugetul e depășit → pipeline degradat care tot răspunde. Singurele două tăceri permise sunt *intenționate*: un om a preluat conversația, sau mesajul e o dublură deja procesată.

Și onestitatea contează: diagrama marchează cu roșu, la vedere, cele două găuri reale cunoscute — excepția neprevăzută în procesare care azi pierde turul tăcut, și mesajul abandonat după prea multe reîncercări de lacăt. Nu sunt ascunse: au card de reparație (NX-140). O diagramă de erori care arată doar ce merge bine ar minți prin omisiune.

Logica — familiile de eșec

- **AI-ul pică** (pană, limită de rată): reîncercare mărginită; tot nimic → triajul rămâne fără verdict, agentul se retrage → plasa finală pune o întrebare de clarificare. Clientul primește ceva, mereu.
- **Validatorul respinge** un răspuns: o reîncercare cu explicația greșelii → tot rău → formulare deterministă fără cifre.
- **Bugetul zilnic e atins:** AI-ul se stinge pentru magazinul respectiv, dar porțile, cache-ul și FAQ-ul funcționează în continuare — răspunsuri degradate, nu tăcere.
- **Infrastructura:** Redis căzut la recepție → 503, Meta reîncearcă (nimic pierdut); cache-ul sau FAQ-ul dau eroare → se tratează ca „nu am găsit în cache”, turul continuă; muncitorul moare în mijlocul turului → mesajul rămâne nerevendicat și un „reaper” îl preia din nou. Plus cele două găuri cunoscute, marcate cinstit (vezi mai sus).
- **Procesele de fundal:** un job proactiv care crapă e marcat „failed” cu eveniment (vizibil); un job de mentenanță care crapă se loghează și se reia la următorul interval; poller-ul Telegram care dă eroare doarme 3 secunde și reîncearcă.
- **Livrarea:** trimitere eșuată → reîncercări cu pauze crescânde → epuizate → mesajul e marcat „mort”, vizibil în date. Iar dacă dispecerul moare între „am luat mesajul” și „l-am marcat trimis”, mesajul reapare automat după un timeout de vizibilitate.

DE REȚINUT

Diferența dintre un sistem de jucărie și unul de producție nu e că al doilea nu eșuează — e că al doilea **știe dinainte cum eșuează** și a ales pentru fiecare eșec cel mai puțin rău deznodământ posibil.

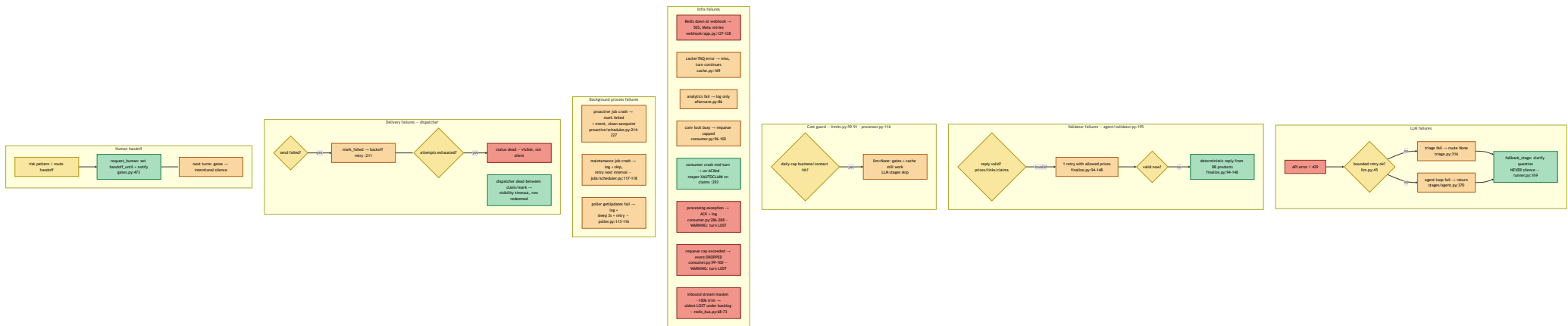


Diagrama 9 — Catalogul eșecurilor și planul pentru fiecare. Diagramă lată: dă zoom pe familia care te interesează.

Diagrama 10 — Mesajele proactive: când noi scriem primii

Ce arată

Singura zonă în care sistemul **inițiază** conversația, nu răspunde: coș abandonat, produs revenit în stoc, actualizări de livrare. Lanțul complet: măturătorii care detectează ocaziile → joburile programate → motorul care le revendică → poarta de conformitate în trei trepte → cutia de ieșire.

De ce am construit-o așa

DECIZIA DE ARHITECTURĂ

Aici sunt cele mai reglementate decizii din tot sistemul — și de aceea zero AI, 100% cod

determinist. Când clientul ne scrie, avem voie să răspundem. Când NOI scriem primii, intrăm pe teritoriul regulilor Meta (fereastra de 24 de ore, template-uri aprobate) și al consimțământului (GDPR). O greșeală aici nu înseamnă un răspuns prost — înseamnă număr de WhatsApp banat sau amendă. Deciziile cu consecințe legale nu se lasă pe seama unui model care „de obicei nimerește”: fiecare regulă e cod care se citește, se testează și se auditează.

Logica, pas cu pas

1. **Măturătorii** (rulați periodic de scheduler) detectează ocaziile: coșuri abandonate, produse revenite în stoc pentru care există abonați → creează joburi programate.
2. **Motorul** revendică joburile scadente atomic (doi muncitori nu pot lua același job) și rezolvă destinatarul: conversația, canalul, identitatea. Nu găsește rută validă → jobul e marcat „failed”, vizibil.
3. **Poarta de conformitate, în trei trepte** — orice treaptă poate opri mesajul:
 - (1) **Consimțământ, pe TIP de mesaj:** mesajele de marketing (coș abandonat, follow-up) cer opt-in de marketing; cele tranzacționale (livrare, revenit în stoc — la care clientul s-a abonat singur) au regulile lor. Fără consimțământ → „skipped”, nu trimis.
 - (2) **Fereastra de 24h:** Meta permite mesaje libere doar în 24h de la ultimul mesaj al clientului. Suntem în fereastră → text liber. Nu suntem →
 - (3) **Template aprobat:** doar șabloane aprobate de Meta, în limba potrivită. Nu există template aprobat → „skipped”. Nu se „improvizază”.
4. **Cutia de ieșire, idempotent:** fiecare job are cheia lui unică — chiar dacă motorul crapă și reia lucrul, mesajul pleacă o singură dată.
5. **Dispecerul** mai face o adaptare la final: canalul suportă template-uri (WhatsApp)? Le folosește. Nu suportă (web/Telegram)? Degradează elegant la text simplu.

Onestitate pe diagramă: două „prize” (actualizarea AWB la expediere și follow-up-ul) sunt definite în cod dar încă necuplate — webhook-ul de comenzi ar trebui să le cheme. Sunt desenate cu roșu și etichetate TODO, ca să nu pretindă diagrama mai mult decât face codul.

DE REȚINUT

Regula de proiectare: **cu cât consecința unei greșeli e mai gravă, cu atât decizia e mai deterministă.** Recomandarea de produs are voie să folosească AI (verificat); trimiterea de mesaje nesolicitate — niciodată.

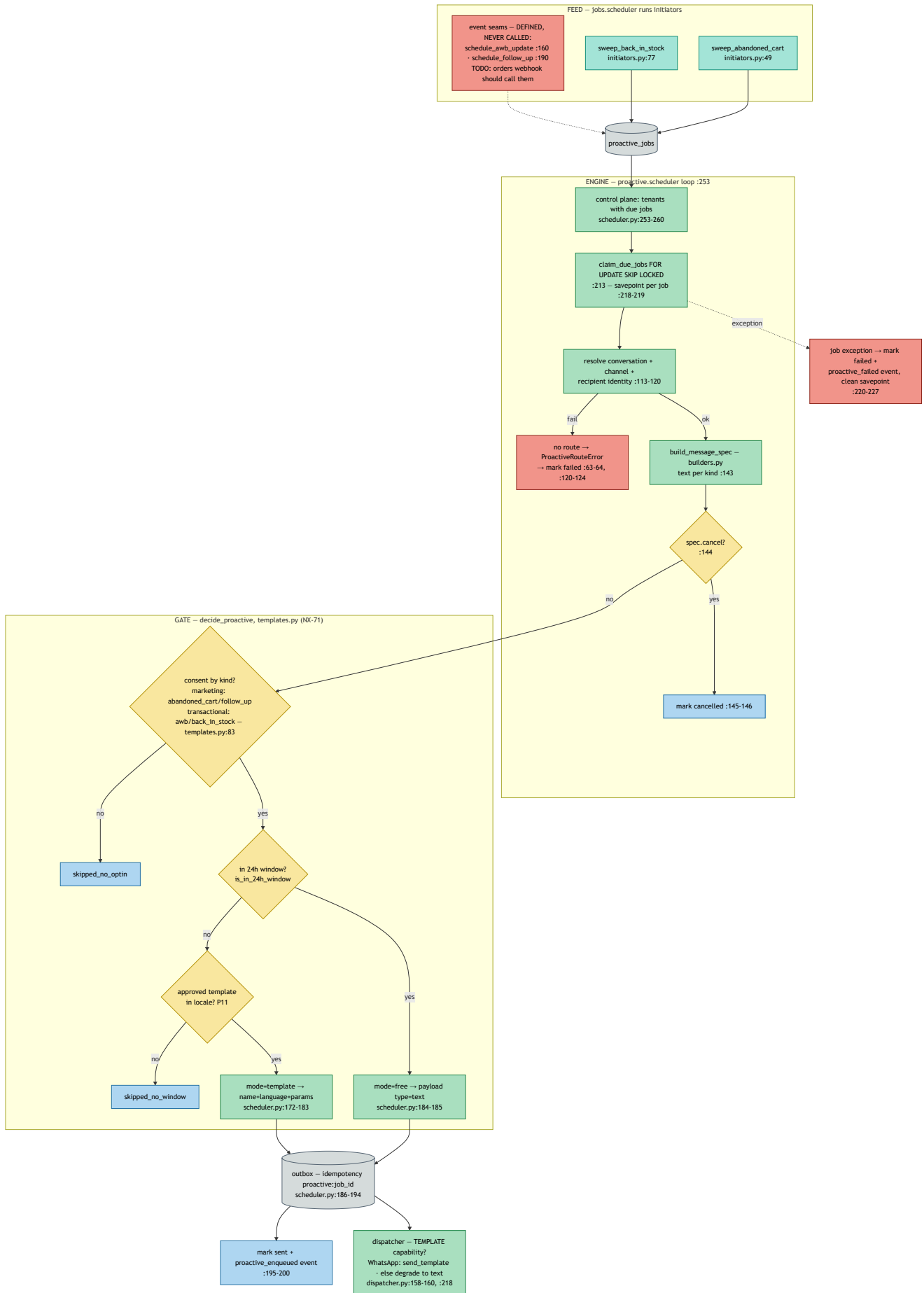


Diagrama 10 — Lanțul proactiv: măturători → joburi → motor → poarta de conformitate → outbox.

În loc de încheiere: lentilele și slăbiciunile asumate

Cele șase lentile

Dincolo de cele 14 diagrame, documentul-mamă mai are un instrument: **lentilele**. Aceleași 11 stagii, privite prin șase întrebări diferite — pentru că deciziile de arhitectură se iau pe întrebări, nu pe topologie:

Lentila	Întrebarea	Răspunsul pe scurt
Cost	Unde se duc banii pe tur?	Exact 10 puncte cheamă un model AI; tot restul e cod. Țintă: 40–60% din trafic servit gratuit.
Date	Cine are voie să scrie ce?	Fiecare câmp din contextul turului are exact UN proprietar. Doi scriitori pe același câmp = arhitectură greșită.
Eșec	Pe unde poate ieși tăcere?	Lanț de degradare explicit; doar două tăceri intenționate (om preluat, dublură).
Flag-uri	Ce rulează de fapt?	86 de întrerupătoare; 71 pornite. Fiecare mecanism nou are kill-switch.
Izolare	Pe unde s-ar putea scurge un tenant?	Trei straturi (cod + rol + RLS); o singură excepție documentată, păzită de un test în CI.
Siguranță	Ce nu are voie să iasă niciodată?	Produce contraindicate (filtre în 5 puncte + plasa finală) și claim-uri medicale (șterse, nu ajustate).

Slăbiciunile cunoscute, pe față

Auditul din care provin diagramele notează și ce NU e încă bine — cu evidență și card de reparație, nu cosmetizat:

- **Cea mai serioasă:** o excepție neprevăzută în procesare pierde turul *tăcut* (clientul nu primește nimic) — încalcă exact principiul „niciodată tăcere”. Reparația are card dedicat (NX-140).
- **Plafonul de scalare #1:** conexiunea la baza de date e ținută ocupată pe durata apelurilor AI → maxim ~10 tururi simultane per proces (NX-141).
- Dispecerul lucrează secvențial, cu pauză de 2 secunde la gol — un tenant lent îi întârzie pe ceilalți.
- Mesajul de rezervă final există doar în română, deși sistemul e RO/HU/EN.

Concluzia auditului: sistemul e neobișnuit de disciplinat — principiile declarate chiar sunt implementate (pipeline liniar, un singur punct de ieșire, validator structural, izolare în straturi). Datoriile reale sunt puține, cunoscute și au plan.

IDEEA FINALĂ

Firul roșu al tuturor celor 14 diagrame e unul singur: **AI-ul propune, codul dispune**. Inteligența artificială e folosită exact acolo unde aduce valoare (înțelegere, căutare, formulare) și e înconjurată de cod determinist peste tot unde o greșeală ar costa bani, încredere sau conformitate legală.